National University
of computer and emerging sciences

# FYP 1 Project Report

# BugLens

Development


**Group Members**

| | |
|---|---|
| Masood Ghauri | 21i-1198 |
| Amna Shahid | 21i-1148 |
| Fatima Qurban | 21i-1195 |

**Supervisor**

Mr. Bilal Khalid Dar

**Co- Supervisor**

Ms. Zoya Mahboob

# Table of Contents

Chapter 1

# Introduction

## Problem Statement and Motivation

The gaming industry is experiencing significant growth, with global revenues reaching about $183.9 billion in 2023 and expected to surpass $200 billion by 2025 [1]. Mobile gaming has emerged as the largest segment, driven by the widespread use of smartphones and tablets, with the global mobile gaming market valued at $108.15 billion in 2022 and projected to grow to around $339.45 billion by 2030, at a compound annual growth rate (CAGR) of 13.55% [2]. Meanwhile, PC and console gaming also remain significant, generating around $43 billion and $63.6 billion in 2023, respectively, and catering to dedicated gamers who invest in high-performance hardware for immersive experiences [1]. With over 3.2 billion gamers worldwide as of 2023, the gaming industry is not just about entertainment but also about community building, with platforms like Twitch enhancing engagement through gaming content and discussions. Additionally, esports and streaming have contributed to the industry's revenue, with the esports market valued at $1.44 billion in 2023 [3] [4].

With the growing gaming industry the game complexity increases as well, with all of this the probability of occurring bugs would rise too. The bug may include minor annoyance to game-breaking glitches and this would severely affect the player's experience. Developers depend on user's feedback, which are reviews, for increasing the user's experience of their application; however managing these bug reports is a challenging task.

Community platforms enable players to discuss and troubleshoot issues collectively, leading to rapid identification of widespread problems. However, the unstructured nature of forum posts and the overwhelming volume of discussions can complicate systematic bug tracking. In-game bug reporting tools offer immediacy and context but may suffer from incomplete descriptions and poor multimedia evidence, limiting their effectiveness.

Digital platforms like Steam centralize game distribution and feedback but present additional challenges due to the vast number of user reviews. These reviews, often lengthy, frequently exceeding 150 words, and filled with jargon, mix critical bug reports with off-topic discussions, making it difficult for testers to accurately identify and prioritize issues. The process of going through these reviews is time-consuming and tedious, increasing the risk of missing crucial bugs and delaying their resolution.

## Problem Solution

To address the challenges, our solution will use AI and automation to streamline bug identification and reporting for games on the Steam platform. It will accept **Steam platform game link or CSV file** containing reviews. For Steam links, our system autonomously scrapes reviews, while for CSV files, it processes and analyzes the data efficiently.

Our **Transformer based AI models** will then identify and **classify bugs into 16 predefined video game bug categories** from the collected reviews. Each bug is systematically categorized, ensuring organized and effective reporting. The system will generate detailed **reports** that include **visualizations and statistic**s, such as charts and graphs, to clearly convey the nature and frequency of the bugs, making it easier for developers to prioritize and address issues.

To sum it up, Automated review collection and bug classification reduces the time and effort compared to manual review processing. Transformer based bug identification model reduces human error as manually going through review one by one we might miss a major bug type. Also the system's flexibility supports both csv based input and steam link, which would directly scrap reviews from the steam. Furthermore, comprehensive visualization helps developers to quickly address the issue in their game site.

## Project Scope

BugLens is a system focusing on automated bug identification and classification giving the facility to generate detailed bug reports for games. Initially the user will have to create an account and after registration they would opt for the most suitable subscription plan. After that, the user would be able to provide a Steam game link or upload a CSV file having reviews of games. Our system would scrape reviews from Steam platform if the user had provided the Steam link. If the user uploaded a CSV file, the system would extract the reviews from the file and perform the classification on them.

Our proposed solution revolves around Transformer Based AI Models for identification and classification of bugs from reviews. The models will perform Sentiment Analysis and overall data analysis of reviews for classification into different categories. There are 16 predefined categories of video game bugs on which the models will be trained for classification.

The reports generated will have summarization of bugs along with data visualization and statistics. The generated reports will be available for download for better understanding of the identified bugs. This will help the developers to pinpoint the issues being detected in the game through reviews.

There will be different subscription plans as well which would offer free and paid tiers. Free tier would  offer a limited number of reports to be generated monthly. The input through link is limited to Steam platform only, while the CSV files can contain reviews from any platform making it easier to identify and detect the bugs present in the games.

## Modules

We have four different modules in our project, which are stated below:

1. ### Dataset gathering and Labeling

   This module consists of scrapping game reviews from “Early Access Games” on STEAM. After scrapping, the data will be pre-processed and labeled on the basis of 16 different bug categories using semi supervised learning techniques.This labeled data will be then provided to fine tune the main Transformer models

2. ### Model Fine Tuning

   This module will revolve around the creation of 16 separate transformer based models fine tuned on different game bug categories. Before passing thorough bug detection models, the reviews will pass through two separate models. One model will do the sentiment analysis, separating the positive and negative bugs. While the other model will do the initial bug identification of the negative reviews.

3. ### Report Generation

   This module will involve generation of bug reports after the classification of reviews into separate categories. The report will involve data visualization regarding the sentiment analysis, bug identification and classification of reviews.

4. User Profile Management and Admin Module

   This module is linked with managing the profile of the users. It would also involve the Reports History Management and managing their subscription plans. Furthermore, this module would also include user's management and displaying system's statistics, in the admin part of this module

## User Classes and Characteristics

| User Class | Description |
|---|---|
| **Admin** | This will be the admin of game bug classification platform, BugLens. The responsibilities of admin would revolve around viewing and managing the users, managing user feedback, and viewing system statistics. |
| **Users** | The users will be the people who want to know about the bugs in specific games. They could either be gamers who wish to have a better gaming experience or they could be the developers who can see what the gaming community thinks about their games and what kind of bugs are common in their games. |
| **FYP Team** | The FYP team will be responsible to fine-tune models for certain outputs, develop a platform for users to generate reports about game bugs and handle deployment and maintenance of whole system |

Chapter 2

# Project Requirements

## Use-Cases

| Use Case 01 | |
|---|---|
| **Use Case** | Create Account |
| **Actors** | User, System |
| **Type** | Primary |
| **Description** | The user will visit the website to use its features. The system prompts the user to create an account. The user provides necessary details (name, email, password), and the system validates the information and creates the account. |

| Use Case 02 | |
|---|---|
| **Use Case** | User Login |
| **Actors** | Users, System |
| **Type** | Primary |
| **Description** | The user will access the login page, provide their credentials (email/username and password), and the system will authenticate the information to grant or deny access. |

| Use Case 03 | |
|---|---|
| **Use Case** | Generate Report from Steam Link |
| **Actors** | User, System |
| **Type** | Primary |

| | |
|---|---|
| **Description** | The user will input Steam Link for a game and the system will scrape reviews from that game, pre-process them, classify them into respective game bug categories and then generate an overall bug report for that specific game. |

| **Use Case 04** | |
|---|---|
| **Use Case** | Generate Report from CSV File |
| **Actors** | User, System |
| **Type** | Primary |
| **Description** | The user will give input in the form of CSV File having game reviews. The system will pre-process the data of CSV File, then classify the reviews into bug categories and generate an overall bug report for that game. |

## Extended Use Cases

### Use Case 01: Create Account

- **Name**: Create Account
- **Scope**: BugLens
- **Level**: User Goal
- **Primary Actor**: User

**Stakeholders and Interests**:

- **User**: Wants to access the features by creating an account.
- **System**: Needs valid account details to ensure proper authentication and user identification.
- User owns a valid Email

**Preconditions:**

**Postconditions:**

- The user account is successfully created, and the user can access the system's features.
- The user's details are stored securely in the system.

**Main Success Scenario:**

| Actor Action | System Responsibility |
|---|---|
| 1. User clicks on "Create Account." | 2. The system displays the account creation form (name, email, password fields). |
| 3. User fills out the required information. | 4. The system validates the input fields for correctness and completeness on frontend |
| 5. User submits the form. | 6. The system processes the input, creates the account, and stores the user data securely.<br>7. The system sends a confirmation email to the user's provided email address. |
| 8. User receives a confirmation message. | |

**Extensions:**

- **A. Missing or Incorrect Information:**
  - The system prompts the user to fill in any missing fields or correct invalid data (e.g., invalid email format).
- **B. Account Already Exists:**
  - The system informs the user that an account with the provided email already exists.

## Use Case 02: User Login

- **Name**: Login
- **Scope**: BugLens
- **Level**: User Goal
- **Primary Actor**: User

**Stakeholders and Interests**:

- **User**: Wants to log in and access features (bug-classification) on the website.

- **System**: Needs valid credentials to authenticate the user and provide secure access.

**Preconditions:**

- The user has an existing registered account on BugLens

**Postconditions:**

- The user is successfully logged in and granted access to their account and features.
- If authentication fails, the system prompts the user to retry or reset their password.

**Main Success Scenario:**

| Actor Action | System Responsibility |
|---|---|
| 1. User navigates to the login page. | 2. The system displays the login form (email/username and password fields). |
| 3. User enters their email/username and password. | 4. The system validates the credentials. |
| 5. User submits the login form. | 6. The system verifies the credentials and authenticates the user. |
| 7. User is logged in successfully. | 8. The system redirects the user to their dashboard or home page. |

**Extensions:**

- **A. Invalid Credentials:**
  - The system displays an error message and prompts the user to re-enter the correct email/username and password.
- **B. Forgot Password:**
  - The system provides a "Forgot Password" option to help the user reset their password via email.

## Use Case 03: Generate Report from Steam Link

- **Name**: Generate Report from Steam Link
- **Scope**: BugLens
- **Level**: System process
- **Primary Actor**: User, System

**Stakeholders and Interests**:

- **User**: Wants to generate a comprehensive bug report for a specific game by providing its Steam link.
- **System**: Needs to accurately scrape, pre-process, classify the reviews, and generate a report based on the game's reviews.

**Preconditions:**

- The user has a valid Steam Link for the game.
- User is Logged in to the BugLens Platform

**Postconditions:**

- A bug report is generated and presented to the user.

**Main Success Scenario:**

| Actor Action | System Responsibility |
|---|---|
| 1. Users input the Steam Link into the system. | 2. The system validates the link and ensures the game's reviews can be accessed.<br><br>3. The system scrapes the reviews from the provided game page. |
| | 4. System pre-processes the scraped reviews.<br>5. The system classifies reviews into the respective game bug categories. |
| | 6. System generates the bug report based on classified data.<br>7. The system presents the user with an overall bug report for the game. |

| | |
|---|---|
| 8. User can view and download the Bug Report | |

**Extensions:**

- **A. Invalid or Inaccessible Steam Link:**
    - The system displays an error message if the Steam Link is invalid or the game's reviews are inaccessible, prompting the user to provide a valid link.
- **B. Scraping Failure:**
    - The system logs the error if review scraping fails due to website changes or network issues and prompts the user to retry later.

## Use Case 04: Generate Report from CSV File

- **Name**: Generate Report from CSV File
- **Scope**: BugLens
- **Level**: System process
- **Primary Actor**: User, System

**Stakeholders and Interests**:

- **User**: Wants to generate a bug report for a game using a CSV file that contains reviews.
- **System**: Needs to correctly process, classify the reviews, and generate an accurate bug report based on the CSV file's data.

**Preconditions:**

- The user has a CSV file containing game reviews under the "review_content" column
- User is Logged in to the BugLens Platform

**Postconditions:**

- A bug report is generated and presented to the user.

**Main Success Scenario:**

| Actor Action | System Responsibility |
|---|---|
| 1. User uploads the CSV file with game reviews. | 2. The system validates the CSV format and content.<br>3. The system pre-processes the data in the CSV. |
| | 4. System classifies the reviews into bug categories. |
| | 5. The system generates an overall bug report based on classified data.<br>6. The system presents the final bug report in a readable format for the user. |
| 7. User views the bug report. | |

**Extensions:**

- **A. Invalid CSV Format:**
    - The system detects an incorrect format or missing fields in the CSV file and prompts the user to upload a valid file.

# Functional Requirements (FRs)

## 1. Module: Dataset gathering and Labeling

**Data Gathering:**

- The team shall manually scrape early access game reviews from Steam.

**Data Labeling:**

- The team shall preprocess the data.
- The team shall label the reviews into the 16 predefined categories for semi-supervised learning.

## 2. Module: Model Fine Tuning

**Sentiment Analysis and Bug Identification:**

- The system should be able to do the initial sentiment analysis of reviews classifying them into positive and negative reviews.
- The system should be able to identify whether the negetive reviews have bugs or not.

**Classification Models:**

- The system should be able to binary classify the reviews into separate predefined categories.

## 3. Module: Data Visualization and Report Generation

**Link-Based Bug Report Generation:**

- The system shall enable user to input a link to a specific game on the Steam platform to generate bug report

**CSV File Upload for Bug Report Generation:**

- The system shall be able to provide the user with the option to upload a CSV file containing game reviews.

**Bug Report:**

- The system shall enable users to generate reports having classification of reviews in different categories
- The system shall enable users to view data visualizations related to sentiment analysis.
- The system shall enable users to view data visualizations related to bug identification.
- The system shall enable users to export/download reports in multiple formats, such as PDF or Word.

4. Module: User Profile Management and Admin Module

1. User Profile Management

**User Profile**

- The system shall allow users to view and update their personal details.
- The system shall ensure that user details are securely stored by being accessible only to authorized users.

**Previous Reports Management**

- The system shall provide users with access to their previously generated reports.
- The system shall allow users to download, view, or delete their previous reports.

**Subscription Management**

- The system shall allow users to manage their subscription plans.
- The system shall notify users of upcoming subscription renewals, expirations, or changes in subscription status.

2. Admin Module

**User Management**

- The system shall allow the admin to manage user accounts.
- The system shall allow the admin to deactivate or reactivate user accounts as needed.

**System Statistics**

- The system shall provide the admin with access to overall system statistics.
- The system shall allow the admin to export system statistics in various formats (e.g., PDF, CSV).

# Non-Functional Requirements (NFRs)

## Security:

The system will use AES-256 encryption for storing the user data.

## Performance:

The system shall process, classify reviews and generate a report within a span of 5-10 minutes, ensuring timely results.

### Compatibility:

- The system shall support file format like csv, to upload for review classification.
- The system shall be able to scrape data from STEAM directly from the link provided by the user.

### Usability:

The UI shall be evaluated using Miller's principle, ensuring that it is Minimal, Intuitive, Consistent, Observable, and Learnable.

# Domain Model

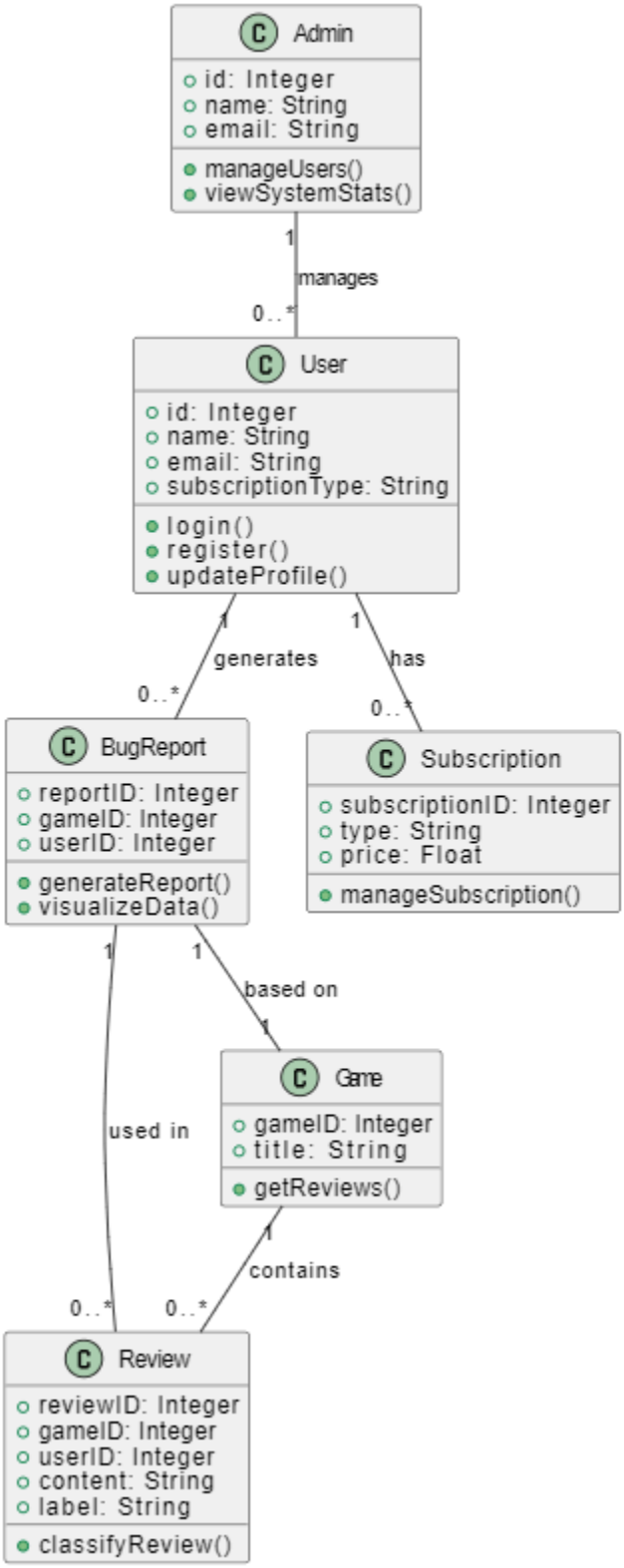



Figure.1 Data Design Diagram for Complete Project

Chapter 3

# System Overview

## UseCase Diagram

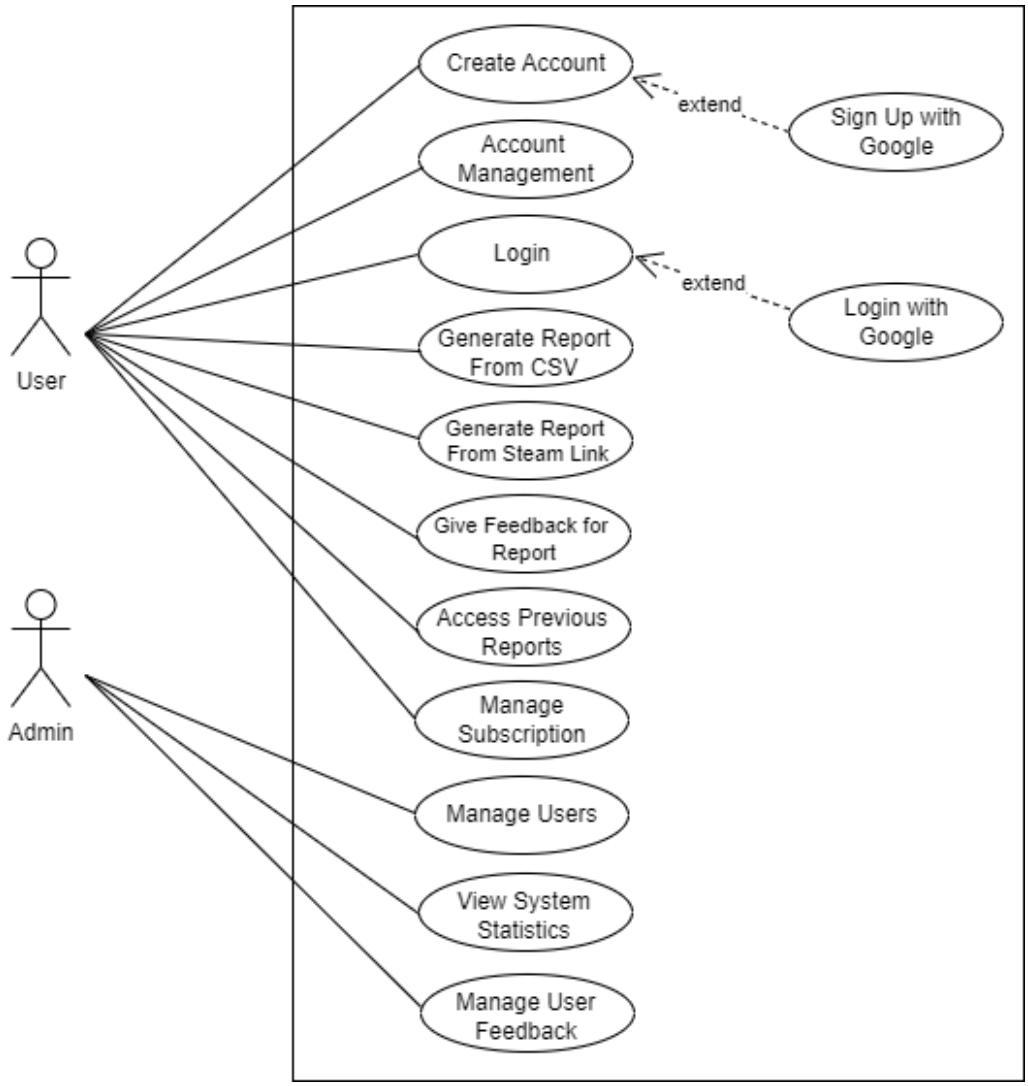



Figure.2 Use Case Diagram for Complete Project

# Data Flow Diagram

1. Level 0

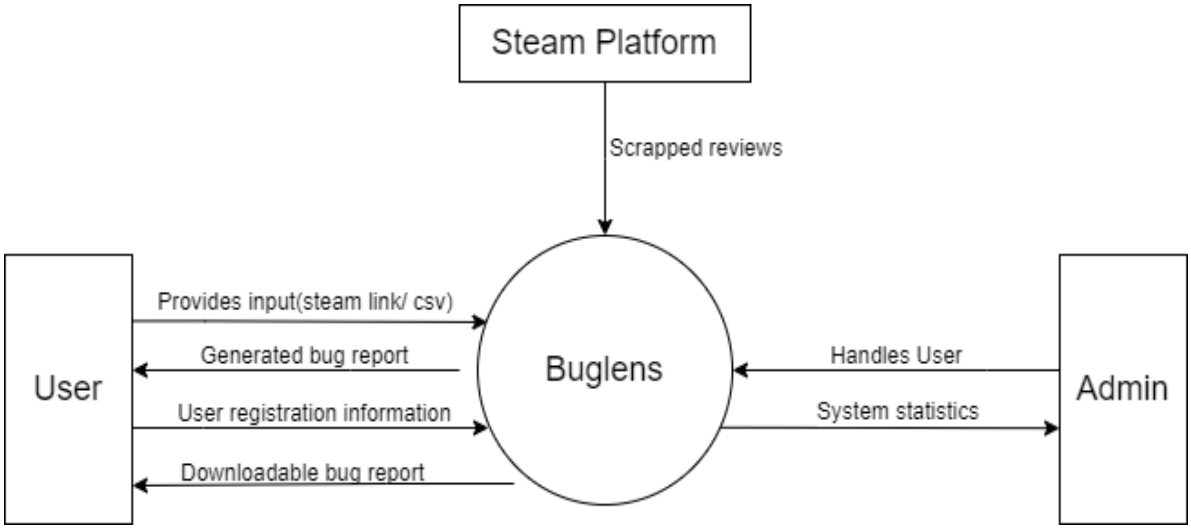
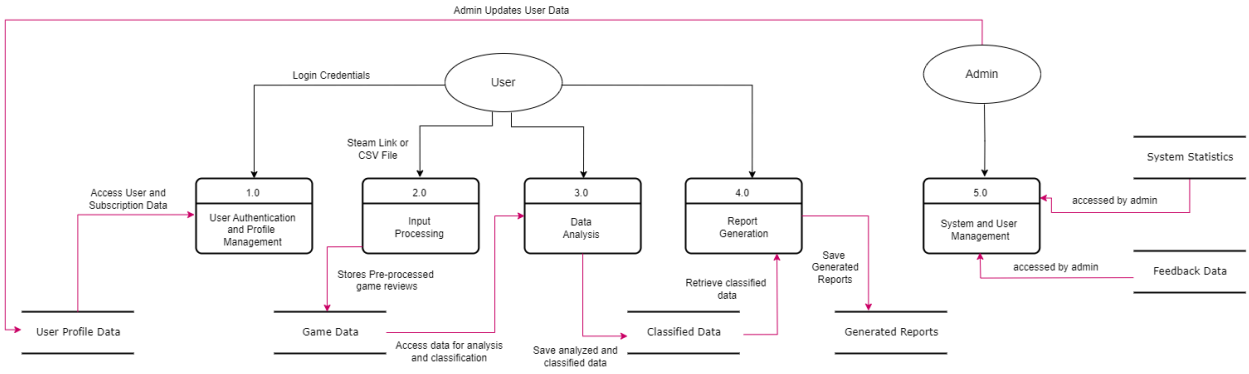
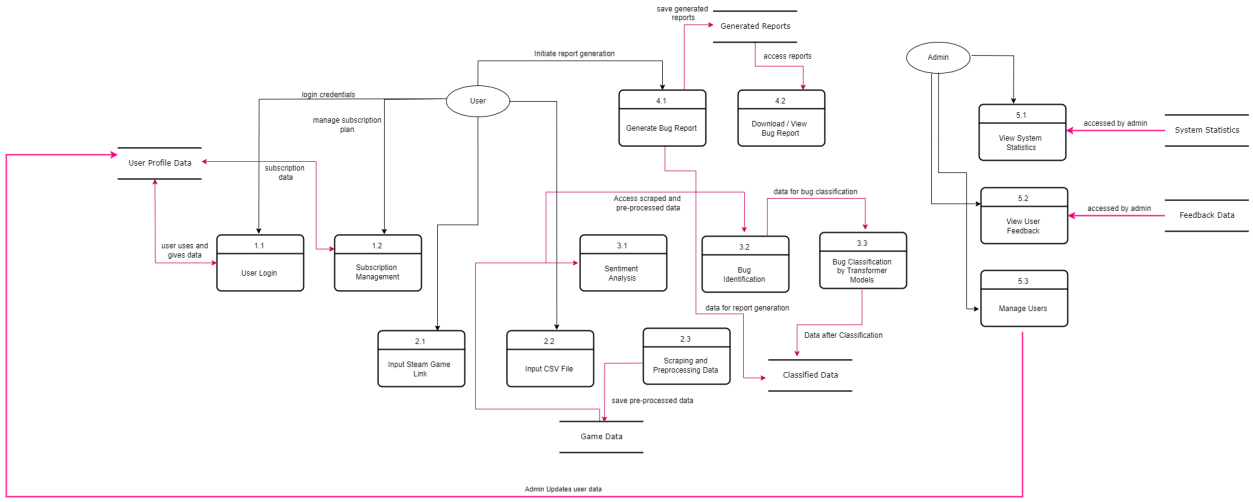



Figure.2 Level 0 DFD

2. Level 1



### 3. Level 2




## Sequence Diagram

### 1. Create Account

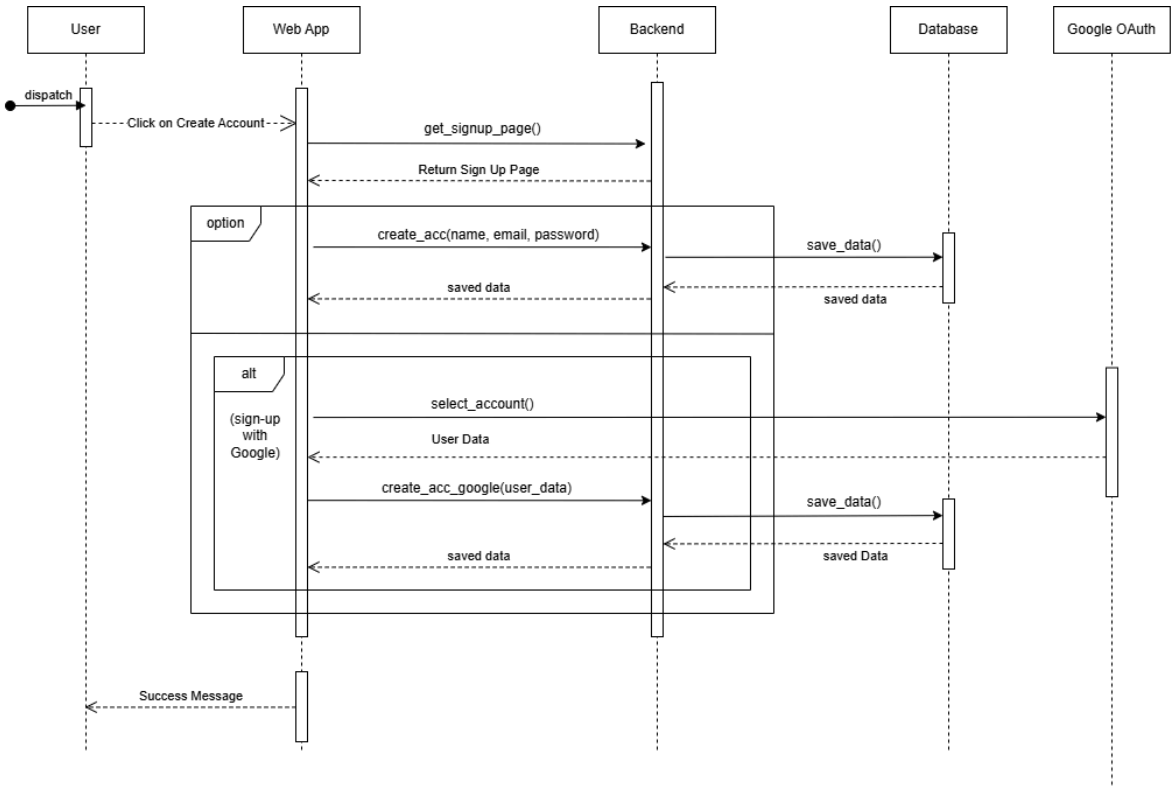



Figure.5 Sequence diagram for “create account”

## 2. User Login

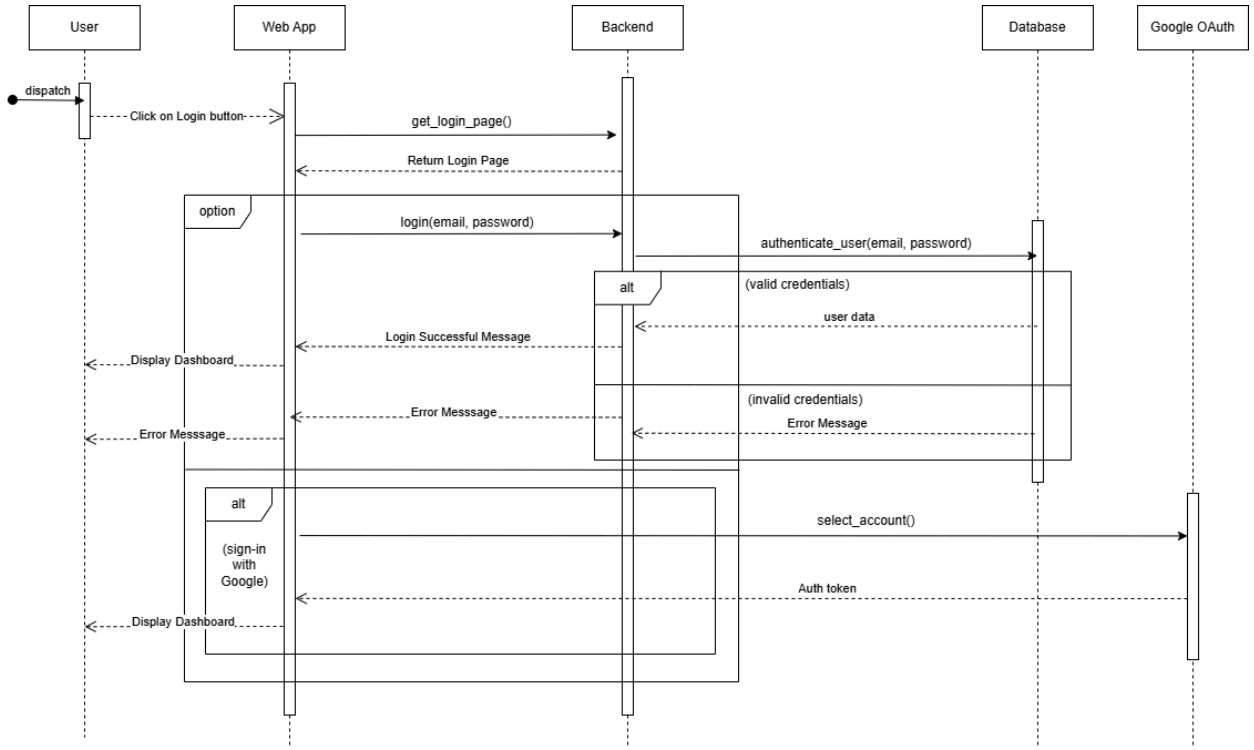



Figure.6 Sequence Diagram for “user login”

## 3. Generate Report from Steam Link

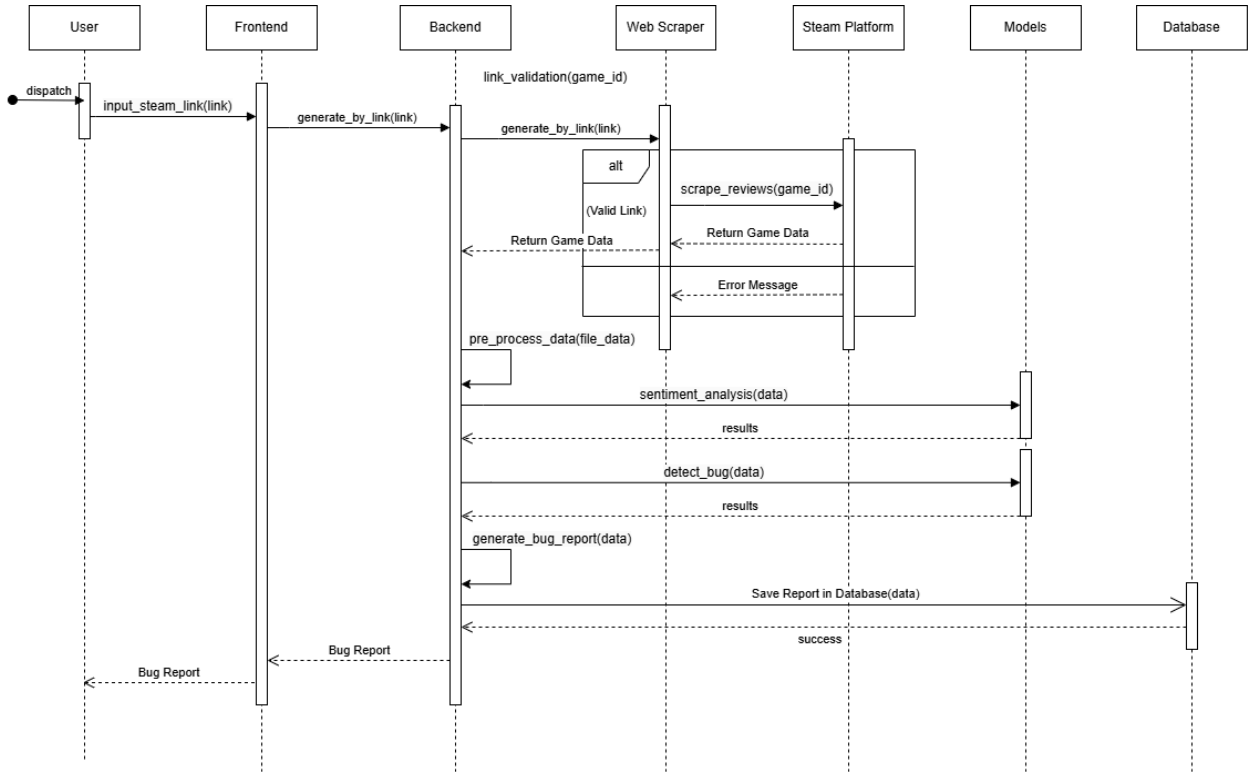



Figure.7 Sequence Diagram for “generate report from steam link”

## 4. Generate Report from CSV File

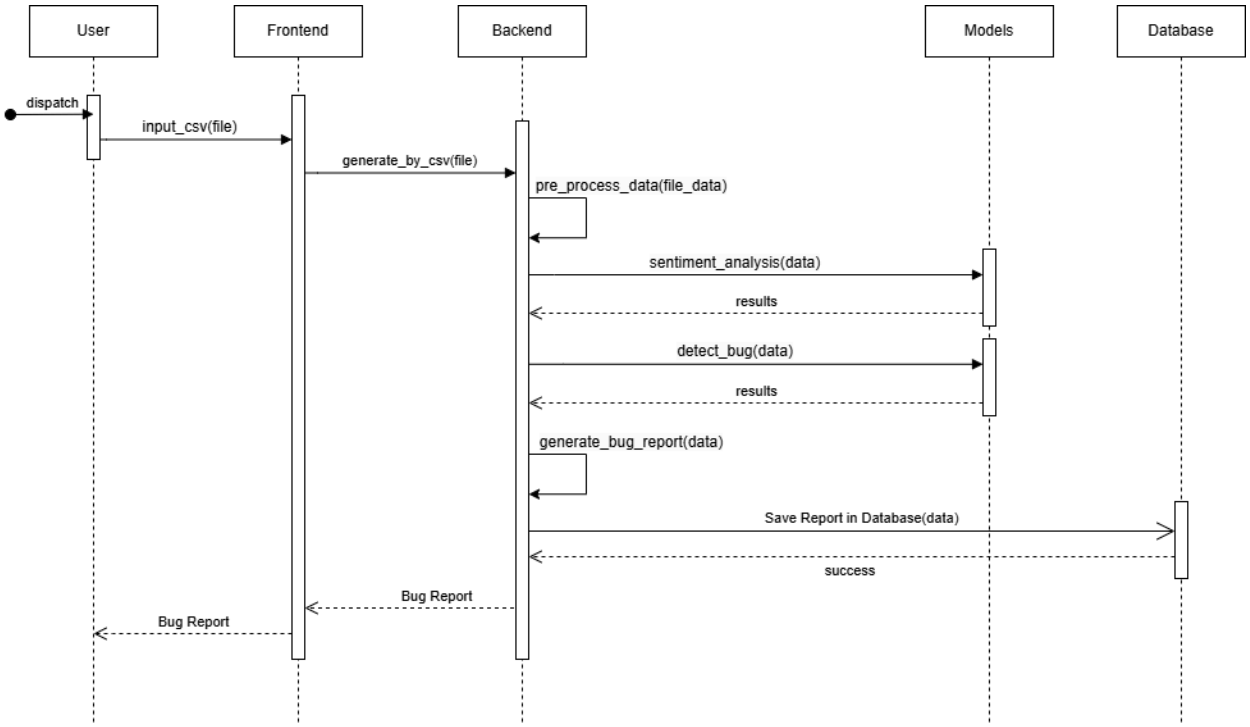



Figure.8 Sequence Diagram For “generate report from csv file”

# Activity Diagram

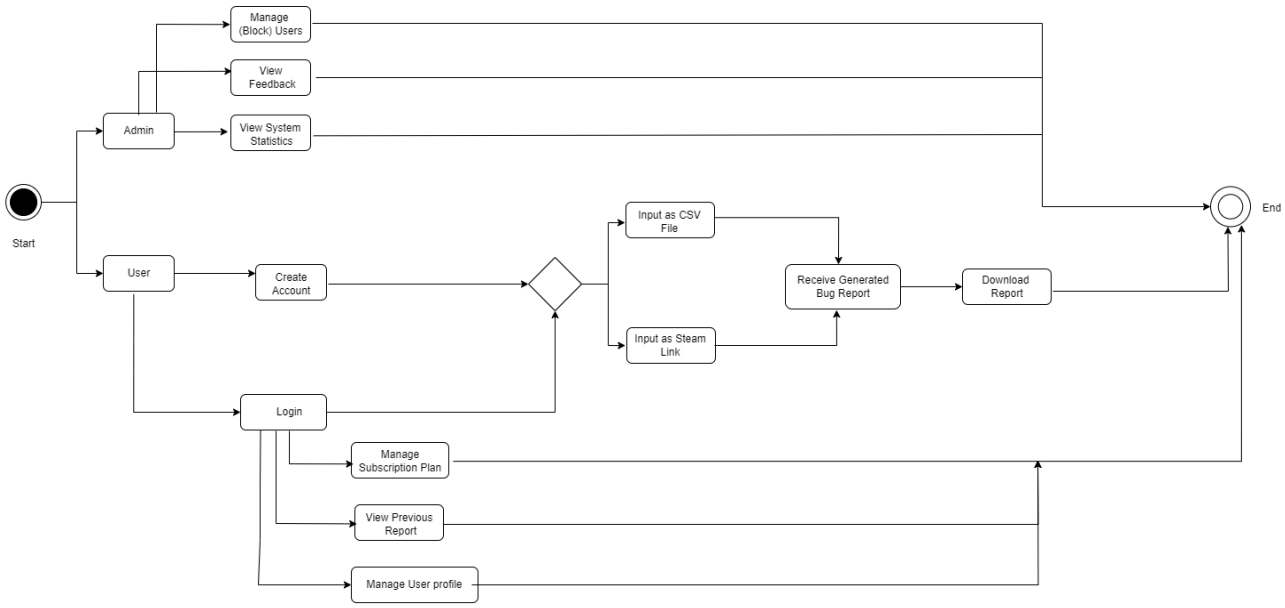



Figure.9 Activity Diagram for Complete Project

# Architecture Diagram

**3-Tier Architecture**

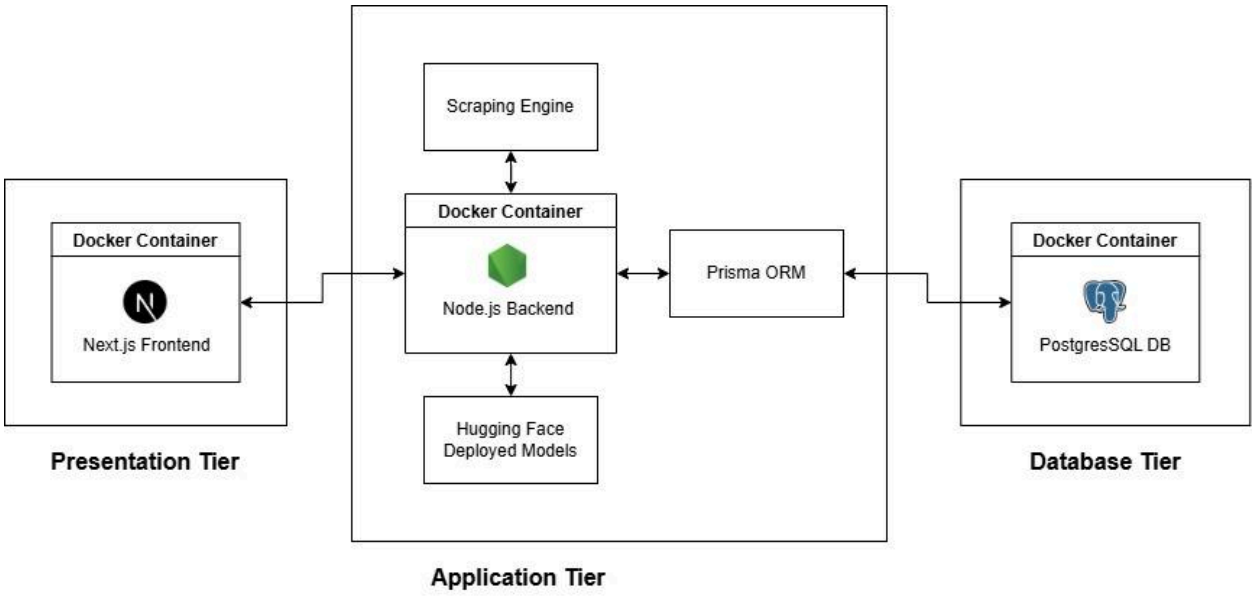



Figure.10 Architecture Diagram

**Reason For Architecture Selection:**

The 3-tier architecture is best for our project because it keeps things simple, with a clear separation between the frontend, backend, and database. This ensures efficient management of our backend-heavy tasks, like data scraping and machine learning, while keeping the frontend responsive. Adding more layers can introduce unnecessary complexity, such as increased communication overhead between layers, slower performance, and more challenging debugging and maintenance, especially since our project doesn’t require additional layers beyond these three core components.

# Data Design

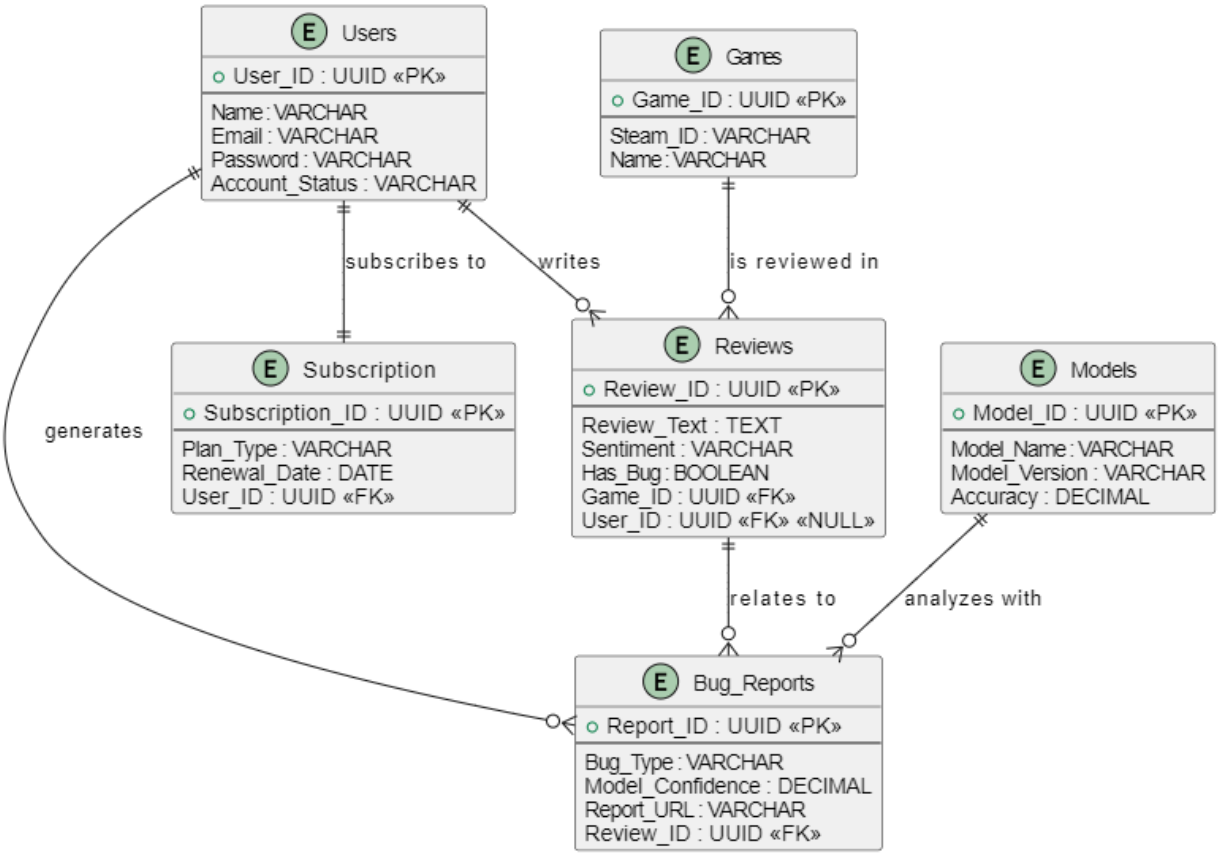



Figure.11 Data Design for Complete Project

# Features achieved in Iteration 1

All the features which were listed under the section “Iteration 1” in the project timeline have been achieved. They along with their Implementation details are given below:

## 1. Data Scraping and Labeling:

- We have scraped a significant amount of reviews from the Steam gaming platform
- After scraping, we pre-processed the data and cleaned the whole dataset.

- The data was split into positive and negative reviews, hasbug and nobug reviews and labeled into 16 predefined bug categories simultaneously.

## 2. Model Fine Tuning:

### 2.1. Sentiment Analysis Model:

We have used Transformer Based Language Models **LongFormer** and **BigBird**. Based on each of these models, we have trained and fine-tuned our sentiment analysis model which initially tells us about the sentiment of a game review either positive or negative. On comparing both models output, we got almost same the accuracy but BigBird Model had less training time with same configurations as of Longformer, so we selected **BigBird** as our final Sentiment Analysis Model

### 2.2. Bug Identification Model:

Similar to the Sentiment Analysis Model, the Bug Identification Model is also made on the basis of two different Transformer Based Language Models, **LongFormer** and **BigBird**. This model detects whether the review is a general statement or does it actually contain a game bug in it. On comparing both models output, we got almost the same accuracy but the BigBird Model had less training time with the same configurations as the Longformer, so we selected **BigBird** as our final Bug Identification.

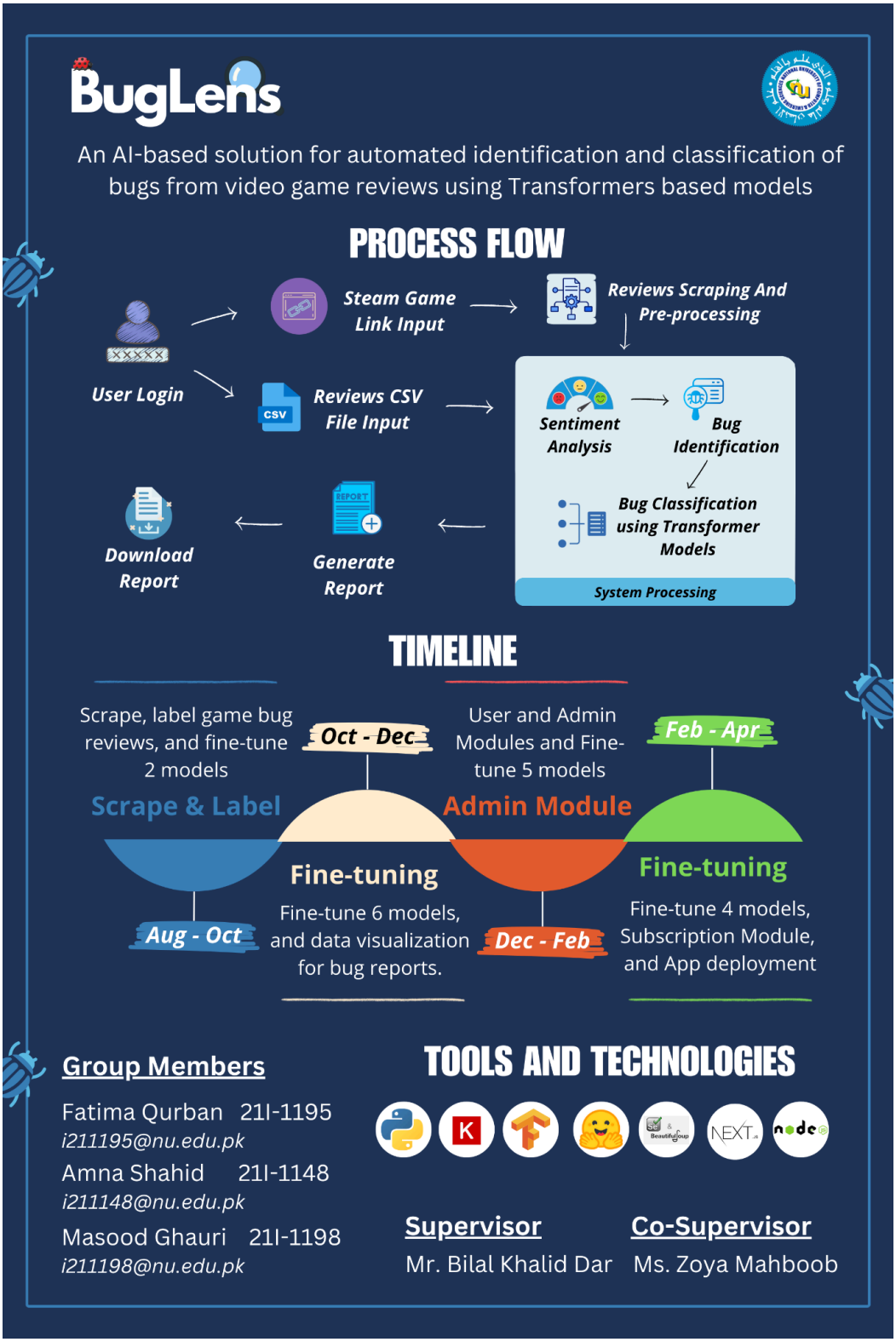
### Reason for Selecting LongFormer and BigBird Models:

The main reason to select LongFormer and BigBird is that they have a very large sequence length which is **4096 tokens**. Given the very **lengthy game reviews**, (average 500 words per reviews) these models are best to handle reviews with large word count. Also, both these models work using a combination of local attention and global attention mechanisms, thus maintaining the context of even lengthy sequences.

## 3. Model Deployment:

We hosted our fine tuned machine learning model on **Hugging Face** that provides an accessible and scalable solution for model inference. The process includes saving the model and tokenizer in the required formats, uploading them to a Hugging Face repository, and making the model available through Hugging Face's Inference API.

Poster


BugLens
An AI-based solution for automated identification and classification of bugs from video game reviews using Transformers based models
PROCESS FLOW
User Login
Steam Game Link Input
Reviews CSV File Input
Reviews Scraping And Pre-processing
Sentiment Analysis
Bug Identification
Bug Classification using Transformer Models
System Processing
Generate Report
Download Report
TIMELINE
Scrape, label game bug reviews, and fine-tune 2 models
Oct - Dec
Scrape & Label
Fine-tuning
Fine-tune 6 models, and data visualization for bug reports.
Aug - Oct
User and Admin Modules and Fine-tune 5 models
Admin Module
Dec - Feb
Feb - Apr
Fine-tuning
Fine-tune 4 models, Subscription Module, and App deployment
Group Members
Fatima Qurban 21I-1195
i211195@nu.edu.pk
Amna Shahid 21I-1148
i211148@nu.edu.pk
Masood Ghauri 21I-1198
i211198@nu.edu.pk
TOOLS AND TECHNOLOGIES
Supervisor
Mr. Bilal Khalid Dar
Co-Supervisor
Ms. Zoya Mahboob

# References


1. *Last looks: The global games market in 2023 | May 2024 Update. (2024). In Newzoo. Newzoo International B.* https://newzoo.com/resources/blog/last-looks-the-global-games-market-in-2023

2. *Mobile Gaming Market Size, Growth Drivers | Forecast - 2030*. (n.d.). Retrieved August 24, 2024, from https://www.fnfresearch.com/mobile-gaming-market

3. *How Many Gamers Are There? (New 2024 Statistics)*. (n.d.). Retrieved August 24, 2024, from https://explodingtopics.com/blog/number-of-gamers

4. *Newzoo's Global Esports & Live Streaming Market Report 2022 | April 2022 Update. (2022). In Newzoo. Newzoo International B.* https://newzoo.com/resources/trend-reports/newzoo-global-esports-live-streaming-market-report-2022-free-version